

Loops

why loops, entry vs exit control, for, while, do-while, break and continue

Some tasks need the same code to run again and again. Printing 1 to 100, checking every character in a name, adding up a list of marks. Writing that code 100 times is not an option. A loop runs a block of code repeatedly, as many times as you need.

DEFINITION

A loop is a control structure that **repeats a block of code** as long as a condition stays true.

Why Loops?

- They avoid writing the same code over and over.
- They handle repetition of any size, whether 10 times or 10,000.
- They keep code short, clean, and easy to change in one place.
- They can repeat a fixed number of times, or keep going until a condition is met.

WITHOUT A LOOP VS WITH A LOOP

```
// without a loop
cout << 1; cout << 2; cout << 3; cout << 4; cout << 5;

// with a loop
for (int i = 1; i ≤ 5; i++) {
    cout << i;
}
```

Types of Loops

Loops are grouped by **when the condition is checked**, either before the code runs or after.

TYPE	CONDITION CHECKED	LOOPS IN C++
Entry Controlled	Before the block runs	for, while
Exit Controlled	After the block runs	do-while

ENTRY CONTROLLED

- The condition is checked first, before running the block.
- If the condition is false at the start, the block runs **zero** times.
- `for` and `while` are entry controlled.

EXIT CONTROLLED

- The block runs first, then the condition is checked.
- So the block always runs **at least once**, even if the condition is false.
- `do-while` is exit controlled.

ENTRY CONTROLLED

check condition

|

true → run block

|

repeat

EXIT CONTROLLED

run block

|

check condition

|

true → repeat



The difference matters in one case: when the condition is false from the very start. An entry controlled loop skips the block entirely. An exit controlled loop still runs it once, because it checks only after.

for Loop

DEFINITION

A **for** loop repeats a block a **known number of times**, with its counter, condition, and update all in one line.

SYNTAX

SYNTAX

```
for (start; stop; step) {  
    // code to repeat  
}
```

The three parts inside the brackets each have one job:

PART	JOB	RUNS
start	Sets up the counter	Once, at the beginning
stop	The condition to keep going	Before every repeat
step	Updates the counter	After every repeat



WHEN THE NUMBER OF ITERATIONS IS KNOWN

Use a **for** loop when you know in advance how many times to repeat, such as 1 to 10, or once per item in a list.

MAIN.CPP

```

for (int i = 1; i ≤ 5; i++) {
    cout << i << " ";
}
// output: 1 2 3 4 5
  
```

Step through what happens each time:

I	I <= 5 ?	PRINTS
1	true	1
2	true	2
3	true	3

I	I <= 5 ?	PRINTS
4	true	4
5	true	5
6	false	loop stops

INFINITE FOR LOOP

- A loop that never stops is an infinite loop.
- It happens when the condition never becomes false.
- Leaving all three parts empty, `for(;;)`, also loops forever.
- Usually a bug, most often a forgotten or wrong `step`.

INFINITE LOOPS

```
for (int i = 1; i ≤ 5; ) {  
    cout << i;    // i never changes, so i ≤ 5 is always true  
}  
  
for (;;) {  
    // no condition at all, runs forever  
}
```

 The most common cause is forgetting the `step`. If the counter never updates, the condition never turns false, and the loop runs forever. Always make sure the counter moves toward the stopping point.

break and continue

These two keywords give you control from inside the loop body.

KEYWORD	WHAT IT DOES
break	Exits the loop immediately, even if the condition is still true.
continue	Skips the rest of this turn and jumps to the next repeat.

BREAK

MAIN.CPP

```
for (int i = 1; i ≤ 10; i++) {  
    if (i == 5) break;    // stop when i reaches 5  
    cout << i << " ";  
}  
  
// output: 1 2 3 4
```

CONTINUE

MAIN.CPP

```
for (int i = 1; i ≤ 5; i++) {  
    if (i == 3) continue; // skip only 3  
    cout << i << " ";  
}  
  
// output: 1 2 4 5
```



Easy way to remember: `break` leaves the loop for good, `continue` only skips the current turn and keeps looping.

A `for` loop is perfect when you know the number of repeats in advance. But sometimes you do not, for example, keep asking for a password until it is correct. That is what `while` is for.

while Loop

DEFINITION

A **while** loop repeats a block **as long as a condition is true**, used when the number of repeats is not known in advance.

SYNTAX

SYNTAX

```
while (condition) {  
    // code to repeat  
}
```

- The condition is checked first, so it is an entry controlled loop.
- If the condition is true, the block runs, then the condition is checked again.
- The loop keeps going until the condition becomes false.
- You must update something inside the loop, or it will never stop.

WHEN THE NUMBER OF ITERATIONS IS UNKNOWN

Use a `while` loop when you cannot say in advance how many times it will run, only what should make it stop.

MAIN.CPP

```
int n = 1;

while (n ≤ 5) {
    cout << n << " ";
    n++;           // update, or the loop never ends
}

// output: 1 2 3 4 5
```

The same three ideas from `for` are still here, just spread out:

<code>int n = 1;</code>	→ start, before the loop
<code>while (n ≤ 5)</code>	→ stop, the condition
<code>n++;</code>	→ step, inside the body

INFINITE WHILE LOOP

- Happens when the condition never becomes false.
- Most often caused by forgetting to update the variable inside the loop.
- `while(true)` loops forever on purpose, and needs a `break` to stop.

INFINITE WHILE

```
int n = 1;
while (n ≤ 5) {
    cout << n;    // n never changes, condition always true
}
```

⚠ The trap with `while` is different from `for`. In a `for` loop the update sits in the header where you can see it. In a `while` loop it is buried in the body, so it is much easier to forget.

FOR LOOP VS WHILE LOOP

Both are entry controlled and can do the same job. The difference is which one fits the situation.

FOR LOOP	WHILE LOOP
Use when repeats are known	Use when repeats are unknown
Counter, condition, update in one line	Only the condition in the header
Best for counting and ranges	Best for looping until an event
Update is easy to see	Update is easy to forget

💡 Quick rule: if you can count the repeats before starting, reach for `for`. If you can only describe when to stop, reach for `while`.

do-while Loop

DEFINITION

A **do-while** loop runs the block **first**, then checks the condition, so the block always runs **at least once**.

SYNTAX

SYNTAX

```
do {  
    // statement  
} while (condition);
```

- The block runs before the condition is ever checked, so it is exit controlled.
- After the block, the condition is checked to decide whether to repeat.
- Even if the condition is false at the start, the block still runs once.

AT LEAST ONE EXECUTION

This is the one thing that makes **do-while** special. Compare the two when the condition is false from the very beginning.

CONDITION FALSE FROM THE START

```
int n = 10;

while (n < 5) {
    cout << n;    // never runs, checked first
}

do {
    cout << n;    // runs once, checked after
} while (n < 5);
```

while: check → false → block runs 0 times
do-while: block runs 1 time → check → false → stop

💡 Use **do-while** when the code must run at least once before any check makes sense, like showing a menu and then asking if the user wants to see it again.

SEMICOLON AFTER WHILE

- A **do-while** loop must end with a semicolon after the condition.
- This is the only loop that ends this way.
- Forgetting it is a very common error.

THE SEMICOLON

```
do {  
    cout << n;  
} while (n < 5);    // this semicolon is required
```

⚠ The `for` and `while` loops do not take a semicolon after their header. The `do-while` loop does, right after the closing bracket of the condition. Miss it and the code will not compile.

CODING QUESTIONS

Common beginner problems that loops are used to solve:

PROBLEM	LOOP IDEA
Print 1 to N	for loop from 1 to N
Sum of first N numbers	add each number in a loop
Reverse a number	while loop, peel one digit at a time
Multiplication table	for loop, multiply by 1 to 10
Count digits in a number	while loop until the number becomes 0

EXAMPLE: SUM OF FIRST 5 NUMBERS

```
int sum = 0;

for (int i = 1; i ≤ 5; i++) {
    sum = sum + i;
}

cout << sum;    // 15
```

SUMMARY

- A loop repeats a block of code while a condition stays true.
- Entry controlled (for, while) check first; exit controlled (do-while) checks after.
- Use `for` when repeats are known, `while` when they are unknown.
- `do-while` always runs at least once and needs a semicolon after the condition.
- `break` exits a loop, `continue` skips to the next turn.